Token-Based Clinic Queue: Design Documentation

A queue for walk-ins and appointments at a fictional clinic. All patient data is fictional.

Live app: https://queue-management-system-jet-six.vercel.app (staff `/staff`, display `/display`) · **Source:** https://github.com/zayannnn2000/Queue_Management_System · **Host:** Vercel, with Upstash Redis for shared state.

1. Key design decisions

- **One ordering key, the target time.** Every waiting patient has a single comparable time, so "fair and in order" and "appointments at their scheduled time" become one rule instead of two competing queues.
- **Appointments join the queue only when they check in.** A booking that hasn't arrived can never hold up patients who are present.
- **All queue logic is one pure module** (`lib/queue.ts`): no I/O, and the clock is always passed in. Every staff action goes through a single reducer that the server runs, so the 29 unit tests (including this document's worked example) cover exactly the code the live app uses.
- **The server decides who is next** and writes with a compare-and-set on a version number (a Lua script on Redis), so two staff pressing "Call next" at once can never call the same patient.
- **The wait estimate replays the call-next rule** rather than using a separate formula, so the estimate and the actual order cannot disagree.
- **Explained decisions.** Every call shows who was picked and why (e.g. "A-003 called: appointment at 10:30, the earliest due target").
- **Privacy on the public screen.** The display endpoint only ever sends the token and first initial; full names never leave the staff view.
- **Polling every 3 s** (Next.js API routes on Vercel) instead of WebSockets: simple and robust on serverless, at the cost of up to 3 s lag.

2. Assumptions

- One clinic, timezone Asia/Dhaka; tokens `W-001…` / `A-001…` restart daily.
- 1 counter by default (1–4); any counter serves any patient.
- Grace period 10 min (configurable).
- Default durations: consultation 15, follow-up 10, blood test 5, vaccination 5, prescription renewal 5 min.
- Bookings are for today only, in 5-minute slots, at or after the current time.
- No double-booking: overlapping appointments may not exceed the number of counters.
- No automatic no-shows: overdue bookings are flagged for staff to decide.
- A called patient who doesn't come is a no-show; if they return, they re-join as a new walk-in.
- Only a name and a service are collected. No login, payments or SMS.

3. Queue-handling logic

Patient	Target time
Walk-in	Arrival time
Appointment, checked in by scheduled time + grace (on time or early)	Scheduled time
Appointment, checked in after the grace period	Arrival time: converted to a walk-in, keeps its A-token, flagged in the UI

Call next (per counter):

1. Nobody checked in → the counter stays free.
2. Someone is **due** (target ≤ now) → call the earliest target.
3. Nobody due → call the earliest **upcoming** target, so the counter isn't idle (the only way an early appointment is seen early).
4. Ties → appointment before walk-in, then the lower token number.

Close-out is *Served*, *No-show* or *Served & call next*. Waiting or booked patients can be cancelled or edited until they are called.

4. Waiting-time calculation

Expected duration(service) = mean of the last 5 served visits (called → served), each clamped to [1 min, 3 × default]; none yet → default.
Counter free at = now if idle, else now + max(expected − elapsed, 1 min).
Simulation: repeatedly take the counter free first (time *t*), apply the call-next rule at *t*, start that patient at *t*, and mark the counter free at *t* + expected. **wait = start − now**; position = order in the projection.
Shown as "Due now" when ≤ 2 min, else "~X min" rounded *up* to the next 5.

Worked example: 1 counter, now 10:00, the patient being served is expected to finish at 10:06, and no visit data yet.

Patient	Target	Service	Projected start	Wait	Shown
W-001 walk-in	09:50	Consultation 15	10:06	6 min	~10 min
W-002 walk-in	09:58	Vaccination 5	10:21	21 min	~25 min
A-001 appointment, checked in early	10:05	Follow-up 10	10:26	26 min	~30 min

A-001 does not overtake the two walk-ins, because they have earlier targets. Had W-002 arrived at 10:08 instead, A-001 (target 10:05) would be called first at 10:21. As visits finish, their real durations replace the defaults and every wait is recomputed.

5. Trade-offs and limitations

- When an appointment comes due, a walk-in who arrived after its slot may wait about one extra service. In return, appointments are honoured and nobody is overtaken by a later target. Early appointments do not jump the queue; late ones lose their slot.
- Idle-fill keeps counters busy but can leave a counter occupied just as a later slot comes due. The estimate already includes this.
- The overlap check counts every booking in the window, so it can refuse a slot that would just fit with 2+ counters. It's simple to explain to patients.
- Estimates, not promises: they converge as real durations accumulate, and the averaging dampens one unusually long visit.
- **Limitations:** `/staff` has no login; one queue per deployment; without Redis (local dev) state lives in one process's memory.